



MALLA REDDY COLLEGE OF ENGINEERING & TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY

UNIT I:

Introduction: Algorithm, Psuedo code for expressing algorithms, Performance Analysis- Space complexity, Time complexity, Asymptotic Notation- Big oh notation, Omega notation, Theta notation and Little oh notation, Probabilistic analysis, Amortized analysis.

Divide and conquer: General method, applications-Binary search, Quick sort, Merge sort, Strassen's matrix multiplication.

INTRODUCTION TO ALGORITHM

History of Algorithm

- The word algorithm comes from the name of a Persian author, Abu Ja'far Mohammed ibn Musa al Khwarizmi (c. 825 A.D.), who wrote a textbook on mathematics.
- He is credited with providing the step-by-step rules for adding, subtracting, multiplying, and dividing ordinary decimal numbers.
- When written in Latin, the name became Algorismus, from which algorithm is but a small step
- This word has taken on a special significance in computer science, where “algorithm” has come to refer to a method that can be used by a computer for the solution of a problem
- Between 400 and 300 B.C., the great Greek mathematician Euclid invented an algorithm
- Finding the greatest common divisor (gcd) of two positive integers.
- The gcd of X and Y is the largest integer that exactly divides both X and Y .
- Eg., the gcd of 80 and 32 is 16.
- The Euclidian algorithm, as it is called, is considered to be the first non-trivial algorithm ever devised.

What is an Algorithm?

Algorithm is a set of steps to complete a task.

For example,

Task: to make a cup of tea.

Algorithm:

- add water and milk to the kettle,
- boil it, add tea leaves,
- Add sugar, and then serve it in cup.

“a set of steps to accomplish or complete a task that is described precisely enough that a computer can run it”.

Described precisely: very difficult for a machine to know how much water, milk to be added etc. in the above tea making algorithm.

These algorithms run on computers or computational devices..For example, GPS in our smartphones, Google hangouts.

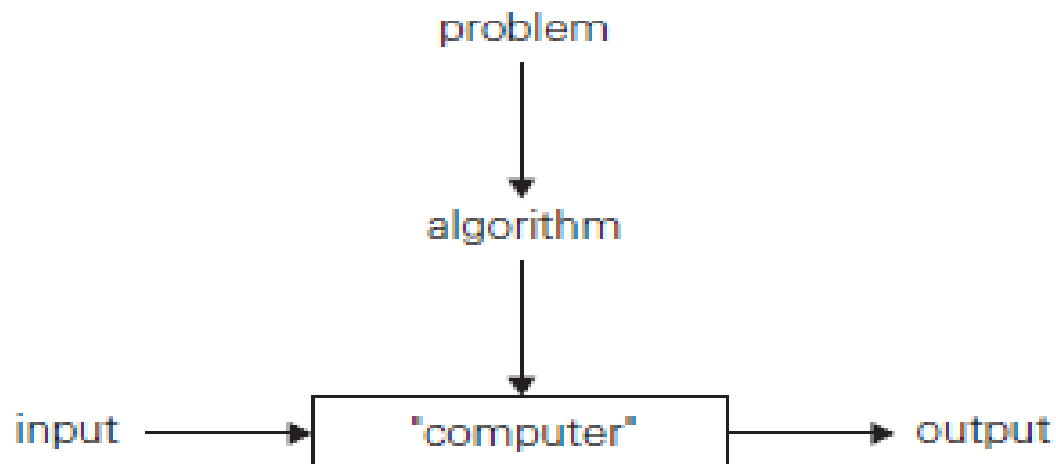
GPS uses shortest path algorithm.. **Online shopping** uses cryptography which uses RSA algorithm.

- Algorithm Definition1:

- An algorithm is a finite set of instructions that, if followed, accomplishes a particular task. In addition, all algorithms must satisfy the following criteria:
 - Input. Zero or more quantities are externally supplied.
 - Output. At least one quantity is produced.
 - Definiteness. Each instruction is clear and unambiguous.
 - Finiteness. The algorithm terminates after a finite number of steps.
 - Effectiveness. Every instruction must be very basic enough and must be feasible.

- Algorithm Definition2:

- An algorithm is a sequence of unambiguous instructions for solving a problem, i.e., for obtaining a required output for any legitimate input in a finite amount of time.
- Algorithms that are definite and effective are also called computational procedures.
- A program is the expression of an algorithm in a programming language



- **Algorithms for Problem Solving**

The main steps for Problem Solving are:

1. Problem definition
2. Algorithm design / Algorithm specification
3. Algorithm analysis
4. Implementation
5. Testing
6. [Maintenance]

- Step1. Problem Definition

What is the task to be accomplished?

Ex: Calculate the average of the grades for a given student

- Step2. Algorithm Design / Specifications:

Describe: in natural language / pseudo-code / diagrams / etc

- Step3. Algorithm analysis

Space complexity - How much space is required

Time complexity - How much time does it take to run the algorithm

Computer Algorithm

An algorithm is a procedure (a finite set of well-defined instructions) for accomplishing some tasks which, given an initial state terminate in a defined end-state

The computational complexity and efficient implementation of the algorithm are important in computing, and this depends on suitable data structures.

- Steps 4,5,6: Implementation, Testing, Maintenance

- Implementation:

Decide on the programming language to use C, C++, Lisp, Java, Perl, Prolog, assembly, etc., etc.

Write clean, well documented code

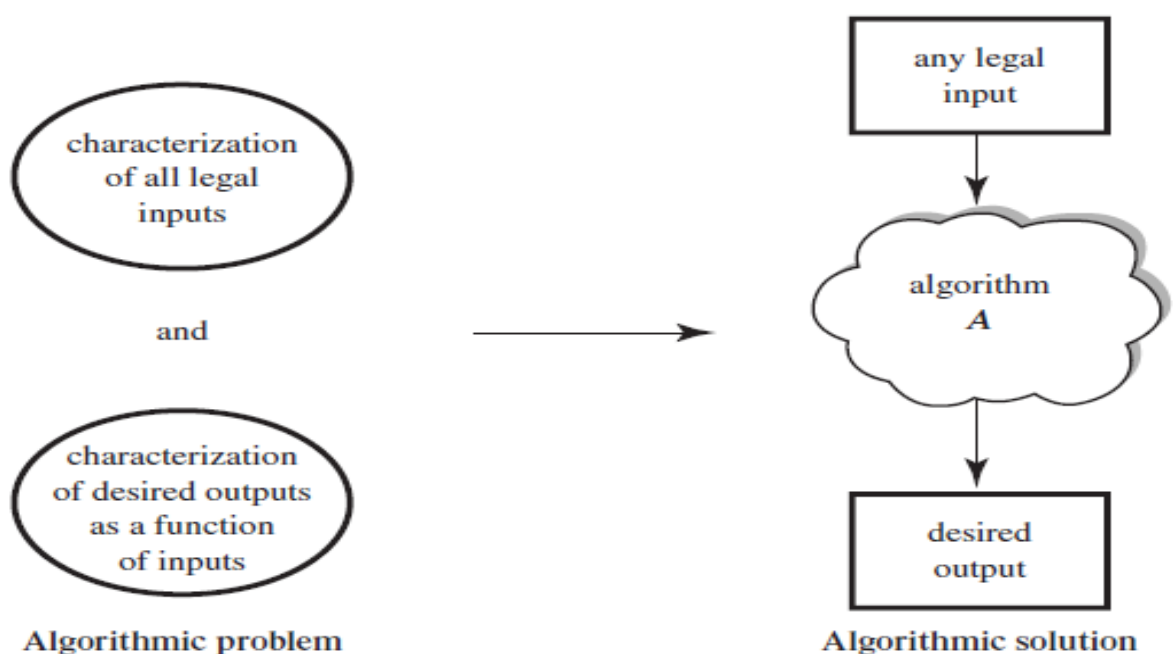
- Test, test, test

Integrate feedback from users, fix bugs, ensure compatibility across different versions

- Maintenance.

Release Updates, fix bugs

Keeping illegal inputs separate is the responsibility of the algorithmic problem, while treating special classes of unusual or undesirable inputs is the responsibility of the algorithm itself.



- **4 Distinct areas of study of algorithms:**

- How to devise algorithms. → Techniques – Divide & Conquer, Branch and Bound , Dynamic Programming
- How to validate algorithms.
- Check for Algorithm that it computes the correct answer for all possible legal inputs. → algorithm validation. → First Phase
- Second phase → Algorithm to Program → Program Proving or Program Verification → Solution be stated in two forms:
- First Form: Program which is annotated by a set of assertions about the input and output variables of the program → predicate calculus
- Second form: is called a specification
- 4 Distinct areas of study of algorithms (..Contd)
- How to analyze algorithms.
- Analysis of Algorithms or performance analysis refer to the task of determining how much computing time & storage an algorithm requires
- How to test a program → 2 phases →
- Debugging - Debugging is the process of executing programs on sample data sets to determine whether faulty results occur and, if so, to correct them.
- Profiling or performance measurement is the process of executing a correct program on data sets and measuring the time and space it takes to compute the results

PSEUDOCODE:

- Algorithm can be represented in Text mode and Graphic mode
- Graphical representation is called Flowchart
- Text mode most often represented in close to any High level language such as C, Pascal → Pseudocode
- **Pseudocode: High-level description of an algorithm.**
- → More structured than plain English.
- → Less detailed than a program.
- → Preferred notation for describing algorithms.
- → Hides program design issues.
- **Example of Pseudocode:**
- To find the max element of an array

```
Algorithm arrayMax(A, n)
Input array A of n integers
Output maximum element of A
currentMax ← A[0]
for i ← 1 to n – 1 do
  if A[i] > currentMax then
    currentMax ← A[i]
```

return <i>currentMax</i>

- Control flow
- if ... then ... [else ...]
- while ... do ...
- repeat ... until ...
- for ... do ...
- Indentation replaces braces
- Method declaration
- Algorithm *method* (*arg* [, *arg*...])
- Input ...
- Output ...
- Method call
- *var.method* (*arg* [, *arg*...])
- Return value
- return *expression*
- Expressions
- Assignment (equivalent to =)
- Equality testing (equivalent to ==)
- n^2 Superscripts and other mathematical formatting allowed

PERFORMANCE ANALYSIS:

- What are the Criteria for judging algorithms that have a more direct relationship to performance?
- computing time and storage requirements.
- **Performance evaluation** can be loosely divided into two major phases:
 - a priori estimates and
 - a posteriori testing.
- ➔ refer as performance analysis and performance measurement respectively
- The space complexity of an algorithm is the amount of memory it needs to run to completion.
- The time complexity of an algorithm is the amount of computer time it needs to run to completion.

Space Complexity:

- Space Complexity Example:
- Algorithm abc(a,b,c)
{
 return a+b++*c+(a+b-c)/(a+b) +4.0;
}

➔ The Space needed by each of these algorithms is seen to be the sum of the following component.

1. A fixed part that is independent of the characteristics (eg: number, size) of the inputs and outputs.

The part typically includes the instruction space (ie. Space for the code), space for simple variable and fixed-size component variables (also called aggregate) space for constants, and so on.

2. A variable part that consists of the space needed by component variables whose size is dependent on the particular problem instance being solved, the space needed by referenced variables (to the extent that it depends on instance characteristics), and the recursion stack space.

The space requirement $s(p)$ of any algorithm p may therefore be written as,

$$S(P) = c + S_p(\text{Instance characteristics})$$

Where 'c' is a constant.

Example 2:

Algorithm sum(a,n)

```
{  
s=0.0;  
for I=1 to n do  
s= s+a[I];  
return s;  
}
```

- The problem instances for this algorithm are characterized by n , the number of elements to be summed. The space needed by 'n' is one word, since it is of type integer.
- The space needed by 'a' is the space needed by variables of type array of floating point numbers.
- This is at least 'n' words, since 'a' must be large enough to hold the 'n' elements to be summed.
- So, we obtain $S_{\text{sum}}(n) \geq (n + s)$
- [n for a[], one each for n, I a & s]

Time Complexity:

- The time $T(p)$ taken by a program P is the sum of the compile time and the run time (execution time)
- The compile time does not depend on the instance characteristics. Also we may assume that a compiled program will be run several times without recompilation. This run time is denoted by $t_p(\text{instance characteristics})$.
- The number of steps any problem statement is assigned depends on the kind of statement.
- For example, comments à 0 steps.
Assignment statements is 1 steps.